Plan de aprendizaje de Claude: reporte de cumplimiento

Sr. Web Developer, AdVise I Aplicación demo Stream I Verificado el 21 de octubre de 2026 con Node 26

30	4	1	2	3	3	1	44
Cumple	Parcial	Lo hace usted	Pendiente	Cambiado por decisión	No hecho (opcional)	Fuera de alcance	Ejercicios

Verificado: 565 pruebas unitarias, 112 pruebas de Playwright con axe (temas claro y oscuro, 320 px), 87 pruebas del backend, tipos, lint con las reglas de accesibilidad de Vue, política de licencias, trazabilidad (47 criterios de aceptación, 17 estados), build de Storybook, presupuesto de rendimiento (JavaScript inicial 80,5 kB gzip de 100), validación del plugin en modo estricto y una auditoría de accesibilidad y rendimiento con el MCP de Chrome DevTools en 10 páginas, escritorio y móvil (accesibilidad 100, buenas prácticas 100, LCP menor a 0,5 s, CLS 0). No verificado: todo lo que necesita a una persona (estados Lo hace usted y Pendiente).

Semana 1 (19 al 23 de oct.): fundamentos y skills

Ejercicio propuesto en el plan	Estado	Evidencia o pendiente
Confirmar que Claude Code funciona con la licencia de Gorilla Logic	Cumple	Confirmado por usted: el trabajo se hace con el Claude de Gorilla Logic.
Montar el sandbox: Vue 3 + TypeScript, Vite, Vue Router, Pinia, Vitest + Vue Test Utils, ESLint con reglas de accesibilidad de Vue, Storybook, Playwright	Cumple	Todo presente. ESLint usa eslint-plugin-vue y eslint-plugin-vuejs-accessibility, y vue/no-v-html es un error. npm run check está en verde.
Escribir el API mock con los cinco comportamientos: preview con retenidos, apply con fallo parcial, rechazo de campo bloqueado, lista de 2.000 filas, límite de tasa	Cambiado por decisión	Se conservan: preview con retenidos y razones, apply con fallo parcial reintentable (incluye fallos de límite de tasa del proveedor), idempotencia. La lista de 2.000 filas y la edición en línea con campo bloqueado se quitaron por su decisión (las reemplazó el filtro de estado en Campaigns); los bloqueos de campo siguen en el formulario de cambio real. mock-backend/ y el API de Laravel.
Shell de la aplicación: rutas, inicio de sesión y sesión, cliente API tipado con un solo lugar para errores, estados loading, empty y error consistentes	Cumple	src/app, src/shared/api/client.ts, inicio de sesión con Sanctum y roles. Cada lista y pestaña tiene estados loading, empty y error.
Lenguaje de diseño pequeño: tokens de color, espaciado y tipografía y componentes base (botón, tabla, campo, badge de estado) con sus estados	Cumple	tokens.json genera las variables CSS; 15 componentes Ds* (incluidos DsTabs y DsAccordion) con historias por estado, claro y oscuro.
Practicar plan mode y permisos	Cumple	Declarado por usted: hecho al construir el proyecto con sus instrucciones.
CLAUDE.md con las convenciones (tokens, estados, fallo parcial, teclado, una prueba por criterio, sin v-html, sin cuentas reales)	Cumple	CLAUDE.md, más reglas en .claude/rules/ (incluida la de UX y arquitectura de la información) y el mapa del producto.
Skill que convierte un spec de pantalla en componente, estados, una prueba por criterio y una historia por estado	Cumple	.claude/skills/screen-from-spec; npm run traceability comprueba cada criterio y estado.

Escribir usted mismo los criterios de aceptación antes de correr el skill	Pendiente	Pendiente: aún no hay tickets. Los specs en docs/specs/ (47 criterios) son borradores ilustrativos, no criterios de un ticket.
Correr el skill en dos specs: un cambio masivo de 300 con preview, retenidos, una aprobación y 12 fallos reintentables; un campo de plantilla de 30 caracteres tras rellenar placeholders	Cumple	Cambio masivo de presupuesto y Ad copy en Campaigns, construidos con el procedimiento del skill. La ejecución en vivo de /screen-from-spec sobre un mockup nuevo se ensaya en prompts/01.
Opcional: lista de 2.000 filas con filtros, orden y columna de estado rápida, con edición en línea que puede rechazarse como bloqueada	Cambiado por decisión	Quitada por su decisión. Campaigns ahora tiene filtro de estado (Active, Paused, Limited, Ended) con conteos; el rendimiento se cubre con el presupuesto de bundle y la auditoría de DevTools.
Revisar todo lo generado y empezar el log de revisión	Cumple	docs/learning-plan/REVIEW_LOG.md, con hallazgos de ejecuciones de revisores independientes.
Asistir al walkthrough de Gorilla Logic Construct	Pendiente	Pendiente: aún no tiene acceso a Construct.

Semana 2 (26 al 30 de oct.): subagentes

Ejercicio propuesto en el plan	Estado	Evidencia o pendiente
Construir un subagente revisor de solo lectura que revise una pantalla contra el lenguaje de diseño, la accesibilidad y los criterios	Cumple	screen-reviewer, más frontend-code-reviewer, backend-code-reviewer (uno por stack), a11y-reviewer y write-path-guardian. Todos de solo lectura.
Plantar cinco pantallas débiles a propósito	Cumple	frontend/review-fixtures/weak-screens/: estilos fijos, sin estado de error, v-html, carga todas las filas y filtra en el navegador, descarta los elementos fallidos.
Ajustar el revisor hasta que atrape las cinco y registrar los cambios	Cumple	El revisor dio FAIL a las cinco nombrando cada defecto; npm run review:fixtures atrapa 5 de 5 por script. Registrado.
Opcional: un cuarto spec (administración de cumplimiento o UTM) generado con el skill y revisado por el revisor	No hecho (opcional)	No construido. El mockup de cumplimiento OEM y el prompt para construirlo en vivo están listos en prompts/10. La página Admin ya lista los rulebooks OEM con fechas efectivas.
Pruebas automáticas de accesibilidad con axe en las pruebas de componente, y una pasada solo con teclado a mano en cada pantalla	Parcial	axe corre en pruebas unitarias, Storybook y Playwright, y el árbol de accesibilidad de DevTools y Lighthouse corren en 10 páginas. Los flujos de teclado están automatizados en Playwright. La pasada a mano es suya.
Hacer de la generación y la revisión un solo paso repetible: el skill genera la pantalla y luego llama al revisor	Cumple	Los pasos 9 y 10 de screen-from-spec llaman al revisor y al log.
Opcional: comparar la historia de cada estado contra el lenguaje de diseño y registrar desvíos	No hecho (opcional)	No hecho.
Chequeo de usabilidad de 15 minutos con un colega en la pantalla de aprobación masiva	Lo hace usted	Necesita a una persona. Guion y tabla en docs/learning-plan/USABILITY_CHECK.md.

Semana 3 (2 al 6 de nov.): hooks, MCP y plugins

Ejercicio propuesto en el plan	Estado	Evidencia o pendiente
Hook que corre lint (con reglas de accesibilidad de Vue), el type check y las pruebas afectadas tras cada edición de componente	Cumple	format.mjs (Prettier, ESLint) y verify.mjs (vue-tsc, vitest related, screen check); probado en tools/hooks.spec.ts.
Hook que bloquea ediciones a archivos de entorno y secretos	Cumple	guard-files.mjs bloquea .env, claves, certificados, credenciales y lockfiles; guard-bash.mjs bloquea comandos destructivos de base de datos.
Conectar el MCP de Playwright, recorrer las pantallas y redactar pruebas e2e con axe, y revisar cada localizador a mano registrando lo rechazado	Parcial	Los servidores MCP de Playwright y Chrome DevTools están en .mcp.json (fijados, gratuitos) y preaprobados en .claude/settings.json. Las 112 pruebas e2e se escribieron como código de Playwright, no se redactaron con el MCP. El recorrido en vivo está en prompts/08.
Lista de 2.000 filas con presupuesto de rendimiento (Lighthouse CI o prueba de tiempos) que rompa el build	Cambiado por decisión	La lista se quitó por su decisión. El presupuesto se mantiene: npm run budget falla si el JavaScript inicial supera 100 kB gzip y npm run audit:devtools falla por debajo de accesibilidad 100, buenas prácticas 95, LCP 2,5 s o CLS 0,1.
Opcional: apuntar una pantalla a un endpoint de Laravel pequeño que devuelva un resultado parcial	No hecho (opcional)	No hecho: las pantallas de lote corren sobre el mock. El resto de la app corre sobre el API real de Laravel (cuentas, campañas, aprobaciones, changelog, campañas simuladas con edición y borrado).
Empaquetar el skill, el revisor y los hooks como plugin personal, instalarlo en un sandbox limpio y mover las convenciones a un skill	Cumple	plugins/stream-frontend-kit v0.2.0: 6 skills, 3 agentes revisores, 2 hooks y 2 servidores MCP; las convenciones viajan como skills generados desde las reglas. Validado en modo estricto e instalado en una configuración vacía.
Correr lint, tipos, pruebas unitarias, Playwright con axe y el presupuesto de rendimiento en un pipeline de CI	Fuera de alcance	Fuera de alcance por su decisión: esta app demo no mantiene pipeline de CI. Todo corre en local con npm run check, npm run e2e, npm run budget y npm run audit:devtools.
Escribir un resumen de una página del setup	Cumple	docs/learning-plan/SETUP_SUMMARY.md, y docs/PRODUCT_OVERVIEW.md para el producto.

Durante todo el ejercicio: revisar la salida de la IA

Ejercicio propuesto en el plan	Estado	Evidencia o pendiente
Conformidad con el lenguaje de diseño: solo tokens y componentes base	Cumple	screen-lint, el revisor de pantallas y ESLint; nada fijo en las pantallas.
Defectos sembrados: romper un componente a propósito y confirmar que una prueba o el revisor falla	Cumple	Se quitaron cuatro salvaguardas a propósito; cada una hizo fallar sus pruebas; restauradas. Registrado.
Accesibilidad: axe en cada componente y flujo, más una pasada solo con teclado a mano	Parcial	axe en ambos temas y a 320 px en cada página; la auditoría de DevTools da 100. La pasada a mano es suya.

Estados y trazabilidad: cada criterio a una prueba, cada estado a una historia	Cumple	npm run traceability: 47 criterios y 17 estados cubiertos.
Seguridad: sin v-html con texto del usuario, sin secretos en el bundle, nada confiado sin leer	Cumple	vue/no-v-html es un error; pruebas de texto plano; el hook de secretos; revisores independientes.
Un log de revisión con al menos un artefacto rechazado o corregido cada semana	Parcial	El log tiene las correcciones y revisiones independientes hasta ahora. Agregue entradas cuando ocurran las semanas 2 y 3.

Agregado después del plan (sus pedidos)

Ejercicio propuesto en el plan	Estado	Evidencia o pendiente
Flujos de las capturas de AdVise y de los criterios del MVP	Cumple	Lanzamiento por cuenta (rulebook OEM, presupuesto propio de Salesforce de la cuenta, retenidos con razones, una aprobación, fallo parcial, recibos), aprobaciones en Campaigns, filtro de estado.
Cuentas con su propia página y presupuesto mensual; las campañas muestran su cuenta	Cumple	/accounts y /accounts/:id (Campaigns, Launch, Changelog), API de cuentas en Laravel con pruebas, enlaces a la cuenta en Campaigns.
Sandbox unido a Campaigns con CRUD completo para campañas simuladas	Cumple	Una pestaña por proveedor, columna Source, crear, editar y borrar; PATCH y DELETE en Laravel con pruebas.
Aprobaciones como tabla con iconos de aprobar y rechazar, dentro de Campaigns	Cumple	Pestaña Approvals con el conteo de pendientes en su etiqueta; la dirección vieja redirige.
Parada de emergencia movida del encabezado a una página Admin (solo admins) con los rulebooks OEM	Cumple	/admin con pestañas Emergency stop y OEM rulebooks; el banner sigue visible para todos mientras las escrituras están detenidas.
Regla de UX y arquitectura de la información; pestañas y acordeones en lugar de contenido apilado	Cumple	.claude/rules/ux-information-design.md, docs/standards/ux.md, DsTabs y DsAccordion, pestaña en la URL.
Revisores separados por stack; skills, reglas, agentes y comandos actualizados; espejo de Codex generado	Cumple	frontend-code-reviewer y backend-code-reviewer; npm run sync:codex con una prueba que falla si el espejo está desactualizado.
Accesibilidad y rendimiento con DevTools, y paquetes actualizados (Node 26, Pest 5)	Cumple	npm run audit:devtools; TypeScript se queda en 6.0 porque typescript-eslint aún no soporta la 7.
Documentación: visión del producto con roles, expectativas por página y diagramas; prompts de demo con mockups; repositorio en GitHub	Cumple	docs/PRODUCT_OVERVIEW.md, prompts/, https://github.com/vanessaaristizabal-gl/demo-stream (commits semánticos en main).